Diamond Dogs

Comparativa de métodos de persistencia: pickle, joblib, json y skops

Criterio	pickle	joblib	json	skops
Facilidad de uso	Muy fácil: `dump` y `load`	Muy fácil: `dump` y `load`	Media: hay que extraer los datos del modelo y reconstruirlo	Fácil, con un paso extra de revisión antes de cargar
Tamaño y legibilidad	Binario, no legible	Binario, no legible; compresión opcional	Texto, muy pequeño en regresión lineal y legible	Archivo comprimido con un `schema.json` que se puede inspeccionar
Seguridad con archivos desconocidos	Mala: puede ejecutar código al cargar	Mala: usa pickle por debajo	Buena: solo carga datos	Buena si se revisan los tipos antes de cargar
Compatibilidad entre versiones	Frágil entre versiones de Python y de scikit-learn	Igual de frágil	Muy buena: no depende de ninguna librería	Ligada a scikit-learn
Guardar la información asociada	Fácil: se guarda un diccionario con el modelo y los datos	Igual que pickle	Muy fácil: todo es un diccionario	Posible, pero con tipos admitidos más restringidos
Instalación con uv	Incluido en Python	Se instala con scikit-learn	Incluido en Python	`uv add skops`

Explicación de los criterios

Facilidad de uso. Con `pickle` y `joblib` se guarda y se carga el objeto entero con una sola instrucción. `skops` funciona igual, pero obliga a decidir qué tipos de objeto se consideran de confianza. Con `json` no se guarda el objeto: hay que escribir a mano qué datos se guardan y reconstruir después el modelo, lo que da más trabajo pero deja el control total.

Tamaño y legibilidad. Los archivos de `pickle` y `joblib` son binarios y no se pueden leer sin cargarlos, que es justo lo que resulta peligroso. Un modelo de regresión lineal solo necesita unos pocos números, así que el archivo `json` es muy pequeño y se puede abrir con cualquier editor de texto. `skops` guarda un archivo comprimido cuyo contenido se puede consultar antes de cargar el modelo.

Seguridad al cargar archivos de origen desconocido. `pickle` y `joblib` pueden ejecutar código arbitrario al cargar el archivo (se explica en el apartado siguiente). `json` solo puede devolver números, textos, listas y diccionarios, por lo que no ejecuta nada. `skops` no ejecuta código mientras no se confíe en los tipos del archivo: permite listar los tipos no confiables con `get_untrusted_types` antes de cargar.

Compatibilidad entre versiones. La guía de persistencia de scikit-learn advierte de que cargar un modelo guardado con otra versión de la librería no está soportado y puede dar errores o resultados incorrectos. Esto afecta a `pickle`, `joblib` y, en parte, a `skops`. Con `json` solo se guardan números y textos, así que el archivo se abre igual con cualquier versión de Python o de scikit-learn.

Información asociada al modelo. La aplicación necesita guardar junto al modelo la fórmula, las columnas de entrada y salida, las métricas de entrenamiento y test y una descripción. En `json` es lo más natural, porque el archivo es un diccionario. En `pickle` y `joblib` se puede guardar un diccionario que contenga el modelo y estos datos. En `skops` es posible, pero hay que revisar que los tipos usados sean admitidos.

Instalación con uv. `pickle` y `json` forman parte de la biblioteca estándar de Python. `joblib` ya está instalado porque scikit-learn depende de él. `skops` requeriría añadir una dependencia nueva con `uv add skops`.

Riesgo de seguridad de pickle y joblib

La documentación oficial de Python avisa de que el módulo `pickle` no es seguro y de que solo se deben cargar datos de origen conocido. El motivo es que un archivo `pickle` no contiene solo datos: contiene instrucciones para reconstruir objetos, y un objeto puede indicar, mediante el método especial `__reduce__`, qué función debe ejecutarse al cargarlo. Cualquiera que pueda modificar o crear un archivo `pickle` puede, por tanto, hacer que se ejecute código en el ordenador de quien lo cargue. `joblib` usa `pickle` internamente, así que tiene el mismo riesgo. La guía de persistencia de modelos de scikit-learn lo recoge como una limitación de seguridad y recomienda cargar solo archivos de confianza.

Ejemplo realizado

El script `prototipos/pruebas_seguridad_persistencia.py` guarda un objeto cuyo método `__reduce__` pide ejecutar `print`. Después carga el archivo con `pickle.load()` sin llamar a `print` en ningún sitio. A continuación hace lo mismo con un archivo `json` que incluye, a propósito, una clave `__reduce__` y un texto con código (`__import__('os').system(...)`). Por último, intenta guardar con `json` el objeto de un modelo `LinearRegression` para mostrar su limitación. Es un ejemplo inofensivo: lo único que hace es imprimir un mensaje. Los archivos se crean en una carpeta temporal que se borra al terminar.

Código del script:

```
import json
import os
import tempfile
import pickle
from sklearn.linear_model import LinearRegression
import numpy as np


with tempfile.TemporaryDirectory() as CARPETA:

    #INTENTO CON PICKLE
    class Ataque:
        def __reduce__(self):
            return (print, (" Código ejecutado al cargar el archivo pickle",))


    print("1) PICKLE")
    ruta_pkl = os.path.join(CARPETA, "ataque.pkl")
    with open(ruta_pkl, "wb") as f:
        pickle.dump(Ataque(), f)
    print("  Archivo guardado. Cargando con pickle.load()...")
    with open(ruta_pkl, "rb") as f:
        pickle.load(f)

    #INTENTO CON JSON
    contenido_malicioso = {
        "datos1": [2.0, -1.5],
        "datos2": 3.0,
        "__reduce__": ["print", "Código ejecutado al cargar el archivo json"],
        "description": "__import__('os').system('echo EJECUTADO')",
    }

    print("\n2) JSON")
    ruta_json = os.path.join(CARPETA, "ataque.json")
    with open(ruta_json, "w", encoding="utf-8") as f:
        json.dump(contenido_malicioso, f)
    print("  Archivo guardado. Cargando con json.load()...")
    with open(ruta_json, encoding="utf-8") as f:
        datos = json.load(f)

    print("  No se ha ejecutado nada. Lo cargado son solo datos inertes:")
    for clave, valor in datos.items():
        print(f"    {clave!r}: {valor!r}  ({type(valor).__name__})")

    # 3) LIMITACIÓN DE JSON CON OBJETOS DE MODELO
    print("\n3) JSON - Limitación: no guarda el objeto del modelo directamente")

    modelo_ejemplo = LinearRegression()
    modelo_ejemplo.coef_ = np.array([[2.5, -1.0]])
    modelo_ejemplo.intercept_ = np.array([10.0])

    ruta_modelo_fallido = os.path.join(CARPETA, "modelo_fallido.json")

    try:
        with open(ruta_modelo_fallido, "w", encoding="utf-8") as f:
            json.dump(modelo_ejemplo, f)
    except TypeError as e:
        print("Demostración del fallo al intentar serializar el objeto directamente:")
        print(f"     TypeError: {e}")
```

Salida del script:

```
1) PICKLE
  Archivo guardado. Cargando con pickle.load()...
 Código ejecutado al cargar el archivo pickle

2) JSON
  Archivo guardado. Cargando con json.load()...
  No se ha ejecutado nada. Lo cargado son solo datos inertes:
    'datos1': [2.0, -1.5]  (list)
    'datos2': 3.0  (float)
    '__reduce__': ['print', 'Código ejecutado al cargar el archivo json']  (list)
    'description': "__import__('os').system('echo EJECUTADO')"  (str)

3) JSON - Limitación: no guarda el objeto del modelo directamente
Demostración del fallo al intentar serializar el objeto directamente:
     TypeError: Object of type LinearRegression is not JSON serializable
```

Resultados

- Con `pickle` aparece el mensaje al cargar el archivo, aunque el código que carga solo llama a `pickle.load()`. Esto demuestra que se ejecutó código incluido en el archivo.

- Con `json` no se imprime ningún mensaje. El contenido "malicioso" se carga como una lista y un texto normales, porque `json` no sabe ejecutar nada.
- Al intentar guardar con `json` el objeto del modelo, falla con `TypeError: Object of type LinearRegression is not JSON serializable`.

Métodos que no ejecutan código y sus limitaciones

- **json.** No ejecuta código. No guarda el objeto del modelo: al intentar guardarlo directamente falla con `TypeError: Object of type LinearRegression is not JSON serializable`, como muestra la tercera parte del ejemplo. Hay que guardar los coeficientes, el término independiente y el resto de datos, y reconstruir el modelo al cargar. Además, aunque no ejecute código, el archivo puede estar mal formado, por lo que al cargar hay que comprobar que están las claves esperadas y que los coeficientes son números.
- **skops.** No ejecuta código mientras no se confíe en los tipos del archivo. Solo sirve para modelos de scikit-learn, añade una dependencia al proyecto y hay que revisar los tipos no confiables antes de cargar.

Elección para Diamond Dogs

Se elige **json**, y los archivos de modelo usarán la extensión `.model.json`, para distinguirlos de cualquier otro archivo JSON.

La seguridad ha sido el criterio decisivo. La aplicación permitirá cargar modelos guardados, y un usuario podría cargar un archivo recibido de otra persona. Con `json` el peor caso es un archivo con datos incorrectos, que se detecta al validarlo, mientras que con `pickle` o `joblib` el peor caso es la ejecución de código ajeno. Además, `json` permite guardar con facilidad la información asociada que pide la aplicación, genera archivos pequeños y legibles, no depende de la versión de Python ni de scikit-learn y no añade ninguna dependencia. La limitación, reconstruir el modelo a partir de los coeficientes, se resuelve con pocas líneas porque un modelo de regresión lineal solo necesita sus coeficientes y su término independiente.

Como el método elegido no ejecuta código al cargar el archivo, no hace falta añadir al `README.md` la advertencia sobre cargar solo modelos de origen conocido.

Estructura de la información guardada

El archivo `.model.json` contiene un único objeto con estos campos:

Campo	Contenido
`version_formato`	Número de versión del formato del archivo (por ejemplo, `1`)
`descripcion`	Texto libre que describe el modelo
`columnas_entrada`	Lista con los nombres de las columnas de entrada, en el orden en que se entrenó el modelo
`columna_salida`	Nombre de la columna de salida
`coeficientes`	Lista de coeficientes, uno por columna de entrada y en el mismo orden
`termino_independiente`	Número con el término independiente
`formula`	Texto con la fórmula del modelo
`metricas`	Objeto con las métricas de `entrenamiento` y de `test`

Cada bloque de `metricas` contiene `filas`, `ecm` y `r2`.

Ejemplo:

```
{
  "version_formato": 1,
  "descripcion": "Valor de la vivienda a partir de ingresos, antigüedad y habitaciones",
  "columnas_entrada": ["median_income", "housing_median_age", "total_rooms"],
  "columna_salida": "median_house_value",
  "coeficientes": [42653.33, 1975.74, 3.85],
  "termino_independiente": -25029.51,
  "formula": "median_house_value = 42653.33 * median_income + 1975.74 * housing_median_age + 3.85 * total_rooms -
      25029.51",
  "metricas": {
    "entrenamiento": {"filas": 16512, "ecm": 6453767350.33, "r2": 0.5172},
    "test": {"filas": 4128, "ecm": 6563178660.01, "r2": 0.4992}
  }
}
```

Pasos para guardar y recuperar un modelo

Guardar

1. Entrenar el modelo con `LinearRegression`.

2. Extraer sus datos: `coef_` e `intercept_`. Como la salida se pasa como DataFrame, `coef_` es `[[c1, c2, ...]]` e `intercept_` es `[t]`, por lo que se toma el primer elemento (`coef_[0]` e `intercept_[0]`).

3. Reunir esos datos con las columnas, las métricas y la descripción en un diccionario.

4. Escribir el diccionario en un archivo `.model.json` con `json.dump`.

Recuperar

1. Leer el archivo con `json.load`.

2. Comprobar que están todos los campos esperados, que los coeficientes son números y que hay un coeficiente por cada columna de entrada.

3. Crear un `LinearRegression` nuevo y asignarle los coeficientes, el término independiente, el número de columnas de entrada (`n_features_in_`) y sus nombres (`feature_names_in_`).

4. Mostrar la información asociada y usar el modelo para predecir.

```
CAMPOS = ["version_formato", "descripcion", "columnas_entrada", "columna_salida",
          "coeficientes", "termino_independiente", "formula", "metricas"]


#Guardar el modelo
def guardar_modelo_json(ruta, datos):
    """Guarda el diccionario de datos del modelo en un archivo JSON."""
    with open(ruta, "w", encoding="utf-8") as f:
        json.dump(datos, f, indent=4, ensure_ascii=False)


def cargar_modelo_json(ruta):
    """Carga y devuelve el diccionario desde el archivo JSON, comprobando que es válido."""
    with open(ruta, "r", encoding="utf-8") as f:
        datos = json.load(f)

    faltan = [c for c in CAMPOS if c not in datos]
    if faltan:
        raise ValueError(f"Archivo de modelo no válido, faltan: {faltan}")
    coeficientes = datos["coeficientes"]
    if not all(isinstance(c, (int, float)) for c in coeficientes):
        raise ValueError("Los coeficientes deben ser números.")
    if len(coeficientes) != len(datos["columnas_entrada"]):
        raise ValueError("Debe haber un coeficiente por cada columna de entrada.")
    return datos

def reconstruir_modelo(datos_cargados):
    """Reconstruye un objeto LinearRegression a partir de los datos cargados."""
    modelo = LinearRegression()
    modelo.coef_ = np.array([datos_cargados["coeficientes"]])
    modelo.intercept_ = np.array([datos_cargados["termino_independiente"]])
    modelo.n_features_in_ = len(datos_cargados["coeficientes"])
    modelo.feature_names_in_ = np.array(datos_cargados["columnas_entrada"], dtype=object)
    return modelo
```

Uso:

```
guardar_modelo_json(ruta, datos_completos)

datos_cargados = cargar_modelo_json(ruta)
modelo_reconstruido = reconstruir_modelo(datos_cargados)
print(datos_cargados["formula"], datos_cargados["metricas"])
print(modelo_reconstruido.predict(x_nuevo))
```

Prueba de concepto

El script `prototipos/prototipo_persistencia.py`, sin interfaz gráfica, se ejecuta con `uv run python prototipos/prototipo_persistencia.py`. Reutiliza el código de la aplicación (`ImportadorDatos`, `separar_datos` y `eliminar_nulos`), la función `escribir_formula` y la configuración de la prueba de concepto de la regresión (`prototipos/prueba_regresion.py`), y el dataset `data/housing.csv`. El archivo del modelo se crea en una carpeta temporal, por lo que no deja archivos en el proyecto:

1. Carga los datos, los separa en entrenamiento (80 %) y test (20 %) con la semilla 42 y elimina las filas con valores inexistentes.

2. Entrena un modelo de regresión lineal con el conjunto de entrenamiento y calcula sus coeficientes, la fórmula y las métricas.

3. Guarda el modelo en un archivo JSON junto con las columnas de entrada y salida, las métricas y una descripción de ejemplo.

4. Carga el archivo, comprueba que es válido, reconstruye el modelo y muestra su información asociada.

5. Comprueba que el modelo reconstruido produce las mismas predicciones que el original, tanto para una entrada nueva como para todo el conjunto de test.

Código del script:

```
import json
from pathlib import Path
import tempfile

import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

from diamond_dogs.importador import ImportadorDatos
from diamond_dogs.preprocesado import eliminar_nulos
```

```
from diamond_dogs.separacion_datos import separar_datos

from prueba_regresion import (
    ARCHIVO, ENTRADAS, SALIDA, PORCENTAJE_TEST, SEMILLA, VALOR_NUEVO,
    escribir_formula)

CAMPOS = ["version_formato", "descripcion", "columnas_entrada", "columna_salida",
          "coeficientes", "termino_independiente", "formula", "metricas"]

#Crear el modelo
def cargar_y_preparar_datos(archivo, entradas, salida, porcentaje_test, semilla):
    """Carga el dataset, separa en train/test y elimina nulos."""
    df, _ = ImportadorDatos(archivo).importar()
    x_train, x_test, y_train, y_test = separar_datos(
        df, entradas, salida, porcentaje_test, semilla=semilla
    )
    x_train, x_test, y_train, y_test, resumen = eliminar_nulos(
        x_train, x_test, y_train, y_test
    )
    return x_train, x_test, y_train, y_test, resumen

def entrenar_modelo(x_train, y_train):
    """Entrena y devuelve un modelo de regresión lineal."""
    modelo = LinearRegression()
    modelo.fit(x_train, y_train)
    return modelo

def calcular_metricas_y_parametros(modelo, x_train, x_test, y_train, y_test, salida, entradas):
    """Calcula métricas, coeficientes, término independiente y fórmula."""
    pred_train = modelo.predict(x_train)
    pred_test = modelo.predict(x_test)

    ecm_train = float(mean_squared_error(y_train, pred_train))
    ecm_test = float(mean_squared_error(y_test, pred_test))
    r2_tr = float(r2_score(y_train, pred_train))
    r2_te = float(r2_score(y_test, pred_test))

    coeficientes = modelo.coef_[0].tolist()
    termino = float(modelo.intercept_[0])
    formula = escribir_formula(salida, entradas, coeficientes, termino)

    return {
        "coeficientes": coeficientes,
        "termino_independiente": termino,
        "formula": formula,
        "metricas": {
            "entrenamiento": {
                "filas": len(x_train),
                "ecm": ecm_train,
                "r2": r2_tr
            },
            "test": {
                "filas": len(x_test),
                "ecm": ecm_test,
                "r2": r2_te
            }
        }
    }


#Guardar el modelo
def guardar_modelo_json(ruta, datos):
    """Guarda el diccionario de datos del modelo en un archivo JSON."""
    with open(ruta, "w", encoding="utf-8") as f:
        json.dump(datos, f, indent=4, ensure_ascii=False)


def cargar_modelo_json(ruta):
    """Carga y devuelve el diccionario desde el archivo JSON, comprobando que es válido."""
    with open(ruta, "r", encoding="utf-8") as f:
        datos = json.load(f)

    faltan = [c for c in CAMPOS if c not in datos]
    if faltan:
        raise ValueError(f"Archivo de modelo no válido, faltan: {faltan}")
    coeficientes = datos["coeficientes"]
    if not all(isinstance(c, (int, float)) for c in coeficientes):
        raise ValueError("Los coeficientes deben ser números.")
    if len(coeficientes) != len(datos["columnas_entrada"]):
        raise ValueError("Debe haber un coeficiente por cada columna de entrada.")
    return datos

def reconstruir_modelo(datos_cargados):
    """Reconstruye un objeto LinearRegression a partir de los datos cargados."""
    modelo = LinearRegression()
    modelo.coef_ = np.array([datos_cargados["coeficientes"]])
```

```
    modelo.intercept_ = np.array([datos_cargados["termino_independiente"]])
    modelo.n_features_in_ = len(datos_cargados["coeficientes"])
    modelo.feature_names_in_ = np.array(datos_cargados["columnas_entrada"], dtype=object)
    return modelo


def main():
    """Coordina todo el flujo usando una carpeta temporal (sin dejar archivos reales)."""
    print("--- 1. ENTRENAMIENTO DEL MODELO ORIGINAL ---")
    x_train, x_test, y_train, y_test, _ = cargar_y_preparar_datos(
        ARCHIVO, ENTRADAS, SALIDA, PORCENTAJE_TEST, SEMILLA
    )
    print(f"Datos preparados: {len(x_train)} train, {len(x_test)} test (Nulos eliminados)")

    modelo_original = entrenar_modelo(x_train, y_train)
    info_modelo = calcular_metricas_y_parametros(
        modelo_original, x_train, x_test, y_train, y_test, SALIDA, ENTRADAS
    )
    print(f"Modelo entrenado. Fórmula: {info_modelo['formula']}")

    with tempfile.TemporaryDirectory() as carpeta_temporal:
        ruta_modelo_temporal = Path(carpeta_temporal) / "modelo_regresion.json"

        print("\n--- 2. GUARDANDO MODELO E INFORMACIÓN EN JSON (EN UNA CARPETA TEMPORAL) ---")
        datos_completos = {
            "version_formato": 1,
            "descripcion": "Modelo de regresión lineal entrenado con datos de vivienda (housing.csv)",
            "columnas_entrada": ENTRADAS,
            "columna_salida": SALIDA,
            **info_modelo
        }
        guardar_modelo_json(ruta_modelo_temporal, datos_completos)
        print(f"Archivo JSON guardado temporalmente en: {ruta_modelo_temporal}")

        print("\n--- 3. CARGANDO Y RECONSTRUYENDO DESDE EL ARCHIVO JSON ---")
        datos_cargados = cargar_modelo_json(ruta_modelo_temporal)
        modelo_reconstruido = reconstruir_modelo(datos_cargados)

        print("Información asociada recuperada del archivo:")
        print(f"  • Descripción: {datos_cargados['descripcion']}")
        print(f"  • Variables de entrada: {datos_cargados['columnas_entrada']}")
        print(f"  • Columna de salida: {datos_cargados['columna_salida']}")
        print(f"  • Fórmula guardada: {datos_cargados['formula']}")
        print(f"  • R² (Entrenamiento): {datos_cargados['metricas']['entrenamiento']['r2']:.4f}")
        print(f"  • R² (Test): {datos_cargados['metricas']['test']['r2']:.4f}")
        print(f"  • ECM (Entrenamiento): {datos_cargados['metricas']['entrenamiento']['ecm']:.2f}")
        print(f"  • ECM (Test): {datos_cargados['metricas']['test']['ecm']:.2f}")

        print("\n--- 4. COMPROBACIÓN DE IGUALDAD EN PREDICCIONES ---")
        nuevo = pd.DataFrame([VALOR_NUEVO])[ENTRADAS]

        pred_original = modelo_original.predict(nuevo)[0][0]
        pred_reconstruida = modelo_reconstruido.predict(nuevo)[0][0]

        print(f"Valores de entrada de prueba: {VALOR_NUEVO}")
        print(f"Predicción del modelo original     : {pred_original:.2f}")
        print(f"Predicción del modelo reconstruido : {pred_reconstruida:.2f}")

        assert np.isclose(pred_original, pred_reconstruida), "¡Las predicciones no coinciden!"

        pred_test_original = modelo_original.predict(x_test)
        pred_test_reconstruida = modelo_reconstruido.predict(x_test)
        print(f"Predicciones comparadas sobre el conjunto de test: {len(x_test)}")

        assert np.allclose(pred_test_original, pred_test_reconstruida), "¡Las predicciones de test no coinciden!"
        print("\nEl modelo cargado produce las mismas predicciones.")


if __name__ == "__main__":
    main()
```

Resultados

Se probó el mismo modelo de regresión múltiple de la comparativa de librerías, con las entradas `median_income`, `housing_median_age` y `total_rooms` y la salida `median_house_value`. Fórmula guardada en el archivo y recuperada al cargarlo:

```
median_house_value = 42653.33 * median_income + 1975.74 * housing_median_age + 3.85 * total_rooms - 25029.51
```

Métrica	Entrenamiento	Test
Filas	16.512	4.128
ECM	6.453.767.350,33	6.563.178.660,01
R^2	0,5172	0,4992

- La información recuperada del archivo (descripción, columnas de entrada y salida, fórmula y métricas) coincide con la del modelo original.
- Predicción para una entrada nueva (`median_income` = 5,0; `housing_median_age` = 20; `total_rooms` = 2.000): 235.454,09 $ con el modelo original y 235.454,09 $ con el modelo reconstruido.
- Las predicciones del modelo reconstruido coinciden con las del original en las 4.128 filas del conjunto de test, por lo que el modelo cargado produce las mismas predicciones.

Dificultades encontradas

Entender por qué pickle es peligroso. Al principio no se entendía cómo un archivo podía ejecutar código sin que el programa lo llamase. Se resolvió con el ejemplo de `__reduce__`, que muestra que `pickle` guarda instrucciones de reconstrucción y no solo datos.

JSON no guarda el objeto del modelo. Al intentar guardarlo directamente da `TypeError`. Se resolvió guardando los coeficientes y el término independiente y reconstruyendo un `LinearRegression` al cargar.

Formato de los coeficientes. Como la salida se pasa como DataFrame, `coef_` e `intercept_` vienen dentro de una lista adicional. Se resolvió tomando el primer elemento (`coef_[0]` e `intercept_[0]`) antes de guardarlos.

JSON seguro no significa archivo válido. Un archivo JSON puede no ejecutar nada y aun así estar incompleto o tener valores incorrectos. Se resolvió comprobando, al cargar, que están todos los campos, que los coeficientes son números y que hay uno por cada columna de entrada.

Aviso de scikit-learn al predecir con el modelo reconstruido. Al reconstruir el modelo a mano, `predict` avisaba de que los datos tenían nombres de columna y el modelo se había entrenado sin ellos. Se resolvió asignando al modelo reconstruido `n_features_in_` y `feature_names_in_` a partir de las columnas guardadas en el archivo.

Archivos de prueba que se acumulaban. Al principio la carpeta de la prueba se creaba con `tempfile.mkdtemp()`, que no se borra sola. Se resolvió con `tempfile.TemporaryDirectory()`, que elimina la carpeta y los archivos al salir del bloque `with`.

Comprobar que las predicciones coinciden. Comparar una sola predicción no demuestra que el modelo reconstruido sea igual al original. Se resolvió comparando también todas las predicciones del conjunto de test con `np.allclose`.

El prototipo y el ejemplo de seguridad se guardan en `prototipos/`, separados del código de la aplicación en `src/`.